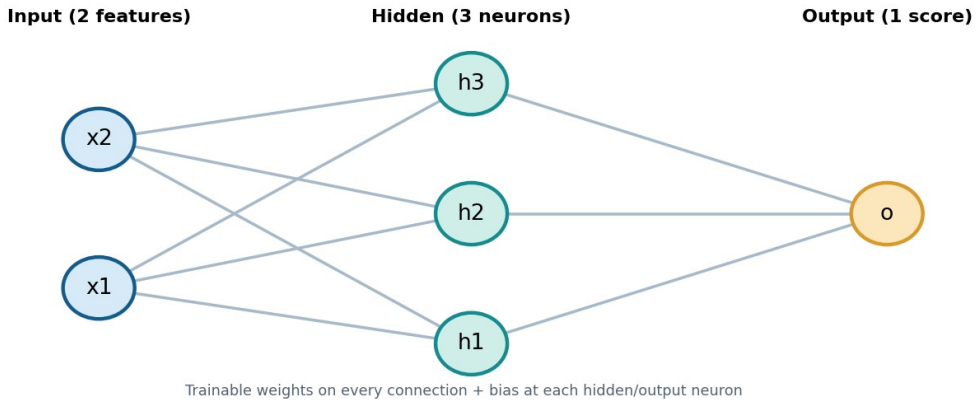
התמחות למידת מכונה ולמידה עמוקה — 5 יחידות לימוד

פרק 12 | רשתות נוירונים עמוקות

מרשת של נוירונים עד לאימון מודל שמזהה דפוסים מורכבים

מהדורת תלמיד • ספר מאוחד יחידה 5 + התמחות • היקף פרק התמחות במפת התכנית: 20 שעות לימוד • כל הדוגמאות והאיורים מקוריים.



איור: מבנה רשת קדמית: שכבת קלט, שכבה חבויה ושכבת פלט.

כיצד להשתמש בפרק

הפרק בנוי כך שכל פעולה מתמטית מקבלת תחילה משמעות מילולית והמחשה. אחר כך נחשב ביד, נבדוק את התוצאה באמצעות NumPy ולבסוף נאמן רשת באמצעות ספריית למידת מכונה. ליד הנוסחאות נפרש כל סימן, ולא נצפה שתלמיד יבין רשת עצבית מתוך קוד בלבד.

הידע הקודם הדרוש: וקטורים ומטריצות, מכפלה פנימית, תקנון נתונים, רגרסיה ליניארית, הפרספטרון, נגזרות, כלל השרשרת ו־Gradient Descent. הפניות לפרקים קודמים מחליפות הוראה כפולה.

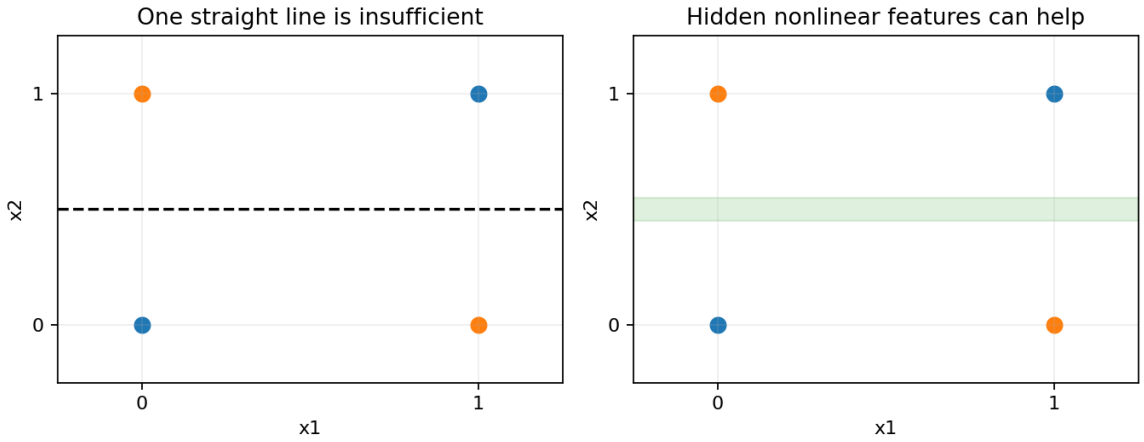
בסיום הפרק התלמיד יוכל...	ראיה לשליטה
להסביר מדוע רשת עם שכבות חבויות שונה ממסווג ליניארי	להמחיש אי־ליניאריות בעזרת XOR
לחשב מעבר קדימה של רשת קטנה	טבלת z, h ופלט עם משקולות נתונות
לחשב נגזרות בצעד Backpropagation	להציג נגזרות למשקולות ולהטיות
לאמן ולבדוק רשת על נתונים חדשים	מחברת עם חלוקה נכונה לנתוני אימון ובדיקה
לפרש גרף שגיאה ולהצביע על מגבלות	פסקת מסקנה מקצועית ומבוססת תוצאות

1. סיפור הפתיחה — מדוע פרספטרון יחיד אינו מספיק?

דודו בונה מערכת המזהה על סמך שני מאפיינים אם קיימת תופעה מסוימת במעבדה. בחלק מהמקרים אפשר להפריד את הדוגמאות בקו ישר: נתונים בצד אחד שייכים למחלקה א׳, ובצד השני למחלקה ב׳. אבל קיימים דפוסים שבהם אותו קו לא יצליח להפריד נכון את כל הנתונים. אם מציירים את ארבע

הדוגמאות של XOR, שתי הנקודות החיוביות נמצאות בפינות נגדיות. אין קו ישר יחיד שיפריד אותן משתי השליליות.

הבעיה איננה שהפרספטרון "התאמן מעט מדי". המגבלה נמצאת בצורה המתמטית של המודל: חישוב סכום משוקלל ולאחריו החלטה באמצעות סף. כדי לתאר גבולות מורכבים יותר נרצה ליצור ייצוגי ביניים חדשים, המתאפשרים באמצעות שכבות חבויות ופונקציות הפעלה לא־ליניאריות.



איור: דוגמת *XOR*: המבנה הגיאומטרי אינו ניתן להפרדה בקו ישר יחיד.

הבהרה מדעית: רשת עם כמה שכבות ליניאריות בלבד עדיין שקולה להעתקה ליניארית או אפינית אחת. אי־הליניאריות של פונקציית ההפעלה היא רכיב חיוני ביכולת לייצג דפוסים לא־ליניאריים.

2. מהי רשת נוירונים רב־שכבתית?

רשת נוירונים היא פונקציה שמורכבת מחישובים רבים המחוברים זה לזה. כל נוירון בשכבה מקבל מספרים מהשכבה הקודמת, מחשב סכום משוקלל ומוסיף הטיה. לאחר מכן הוא מפעיל פונקציית הפעלה. הפלטים של כל הנוירונים בשכבה נעשים לקלט של השכבה הבאה. כך נוצר רצף של ייצוגים מספריים של אותה תצפית.

השכבה הראשונה מקבלת את המאפיינים המקוריים. שכבה חבויה אינה מקבלת ישירות את התווית הנכונה, אלא יוצרת מאפייני ביניים הנלמדים במהלך האימון. שכבת הפלט מחזירה מספר לחיזוי ברגרסיה, או ציונים/הסתברויות בסיווג — בהתאם למבנה הרשת ולפונקציית הפלט.

מבחינת המונח "עמוקה": מקובל לתאר רשת כבעלת מספר שכבות חישוב נלמדות. רשת בעלת שכבה חבויה אחת היא דוגמת ההוראה הפשוטה ביותר לרשת רב־שכבתית; במהלך ההמשך נלמדות גם רשתות שבהן כמה שכבות חבויות.

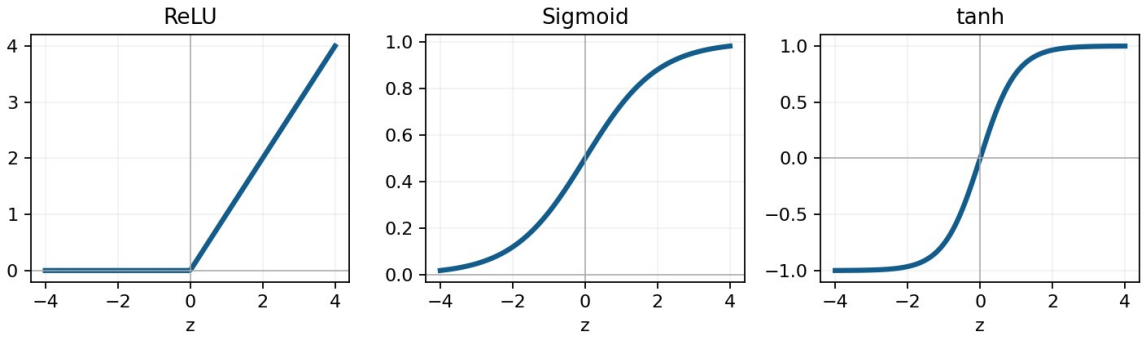
מונח	מה פירושו?	ממה חשוב להיזהר?
נוירון (Neuron)	יחידת חישוב המקבלת קלטים ומחזירה ערך	אין זה תא עצב ביולוגי
משקל (Weight)	מקדם נלמד המגדיל, מקטין או הופך השפעת מאפיין	המשקל אינו בהכרח חשיבות סיבתית
הטיה (Bias)	מספר נוסף המחושב בכל נוירון	אין לבלבל עם הטיה חברתית בנתונים
שכבה חבויה	ייצוג ביניים שנלמד מן הדוגמאות	אין הכרח שאפשר לתת לכל

מונח	מה פירושו?	ממה חשוב להיזהר?
Epoch	מעבר מלא על נתוני האימון	אינו שקול תמיד לצעד עדכון אחד
Batch	קבוצת דוגמאות לעדכון משקולות	גודל ה־Batch משפיע על חישוב ואימון

3. מדוע נדרשת פונקציית הפעלה?

נניח שהשכבה הראשונה מחשבת $z_1 = W_1x + b_1$, והשכבה השנייה מחשבת $z_2 = W_2z_1 + b_2$, בלי שום פעולה לא־ליניארית באמצע. אם מציבים את הביטוי הראשון בשני, מתקבלת עדיין נוסחה מהצורה $Wx + b$. למעשה אפשר לאחד את כל המשקולות למטריצה אחת ואת ההטיות לווקטור אחד. לכן הוספת שכבות ללא פונקציית הפעלה אינה משחררת אותנו מהמגבלה הליניארית.

פונקציית ההפעלה שוברת את הפשטות הזאת. ערכים מסוימים משתנים באופן שונה מאחרים, והרשת מסוגלת ליצור צורות החלטה מורכבות יותר. נדגים שלוש פונקציות נפוצות ונבחין היכן משתמשים בכל אחת.



איור: שלוש פונקציות הפעלה: *ReLU*, *Sigmoid* ו־*tanh*.

```
ReLU(z) = max(0, z)
```

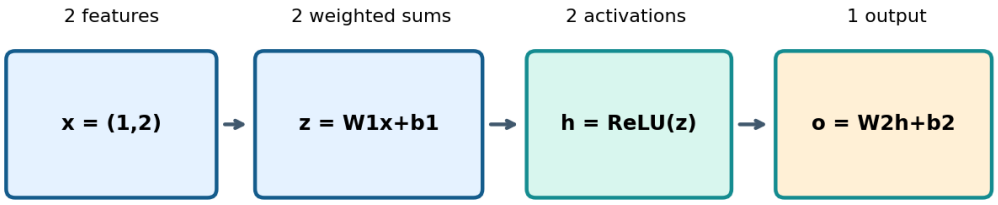
```
sigmoid(z) = 1 / (1 + exp(-z))
```

ReLU מחזירה אפס לכל קלט שלילי ומעבירה קלט חיובי כמו שהוא. Sigmoid ממפה מספר ממשי לערך בין אפס לאחד; כאשר משתמשים בה כפלט של מסווג בינארי ומתאימים פונקציית אובדן מתאימה, התוצאה יכולה להתפרש כהערכת הסתברות. tanh מפיקה ערכים בין מינוס אחד לאחד. הבחירה בפונקציית הפעלה קשורה לתפקיד השכבה וליציבות האימון, לא רק למראה הגרף.

ל־ReLU נגזרת אפס בחלק השלילי ונגזרת אחת בחלק החיובי; בנקודת האפס יש אי־גזירות במובן הקלאסי, וכלי תוכנה בוחרים שם כלל מעשי מתאים. ייתכן מצב שבו נוירונים נשארים כבויים זמן ממושך, ולכן קיימות גם גרסאות אחרות כגון Leaky ReLU — נושא העמקה.

4. כיצד מחשבים מעבר קדימה (Forward Pass)?

המעבר הקדמי הוא סדר החישובים שמפיק תחזית ממשקולות שכבר נקבעו. הוא איננו אימון: במהלך מעבר קדימה רגיל עדיין לא שינינו את המשקולות. נתחיל בדוגמה אחת עם שני מאפיינים, שני נוירונים חבויים ונוירון פלט יחיד. בדוגמה זו הפלט רציף, ולכן נשתמש בו להמחשת רגרסיה.



Forward pass: values flow from left to right

איור: סדר החישוב: קלט → סכום משוקלל → $ReLU$ → פלט.

```
z1 = W1 @ x + b1        h = ReLU(z1)
o = W2 @ h + b2
```

בכתיב זה x הוא וקטור קלט, W1 מטריצת משקולות לשכבה החבויה, b1 וקטור הטיות, z1 הסכומים לפני ההפעלה, h הפלטים אחרי ההפעלה, ו־o תחזית הרשת. סימן @ מציין מכפלה מטריציונית או מכפלה פנימית בהתאם לממדים.

5. דוגמה ידנית מלאה — שני נוירונים ושכבת פלט

דודו מודד שני מאפיינים של מחשב: $x_1=1$ ו־$x_2=2$ (ערכים שכבר הותאמו ליחידות המשמשות בניסוי). נבחר משקולות התחלתיות כדי ללמוד את הפעולה; במודל אמיתי הן נלמדות בזמן האימון ולא נקבעות ידנית לפי התוצאה הרצויה.

```
x = (1, 2)
W1 = [[0.2, 0.4], [-0.3, 0.5]]
b1 = (0.1, -0.2)
W2 = (0.6, -0.4)      b2 = 0.2
```

נוירון חבוי ראשון: מכפילים כל מאפיין במשקל המתאים, מחברים ומוסיפים הטיה. לכן $z_{11}=0.2\times1+0.4\times2+0.1=1.1$. מאחר שהתוצאה חיובית, ReLU מחזירה 1.1.

נוירון חבוי שני: $z_{12}=-0.3\times1+0.5\times2-0.2=0.5$. גם כאן התוצאה חיובית ולכן $h_2=0.5$.

שלב	פעולה	תוצאה
נוירון חבוי 1	0.1 + 2×0.4 + 1×0.2	1.1
נוירון חבוי 2	0.2 − 2×0.5 + 1×0.3−	0.5

שלב	פעולה	תוצאה
ReLU	$\max(0, 1.1), \max(0, 0.5)$	$h = (1.1, 0.5)$
שכבת פלט	$0.2 + 0.5 \times 0.4 - 1.1 \times 0.6$	$o = 0.66$

אם הערך האמיתי הוא $y=1$, פער החיזוי הוא $1-0.66=0.34$. הסימן השלילי אומר שהמודל חזה ערך נמוך מן הערך האמיתי; אין פירושו שמדובר בהסתברות שלילית, שכן בדוגמת הרגרסיה שלנו הפלט אינו מוגבל לטווח 0–1.

6. איך מודדים שגיאה? Loss לעומת מדד הערכה

לפני שנשנה משקולות, צריך להגדיר מה נחשב לטעות. ברגרסיה מקובל להשתמש, בין היתר, בשגיאה ריבועית. בחישוב ידני של תצפית אחת נוח לבחור במחצית ריבוע השגיאה; גורם המחצית מקל על הנגזרת ואינו משנה את נקודת המינימום.

```
L = 0.5 * (o - y)^2
```

```
L = 0.5 * (0.66 - 1)^2 = 0.0578
```

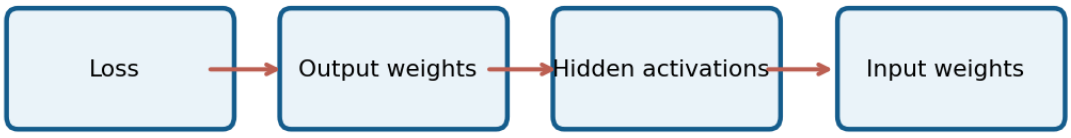
ה־Loss הוא מספר המשמש את תהליך האופטימיזציה. מדדי דיווח כגון MAE, Accuracy ו־F1 אינם בהכרח אותה פונקציה שעליה מתבצע עדכון המשקולות. בסיווג בינארי עם פלט Sigmoid נפוץ להשתמש ב־Binary Cross-Entropy, ולא רק בריבוע הפרש פשוט.

```
BCE = -[ y*ln(p) + (1-y)*ln(1-p) ]
```

בנוסחה זו p היא ההסתברות החזויה למחלקה החיובית, ו־y היא התווית האמיתית שערכה אפס או אחד. כאשר המודל בטוח בתחזית שגויה, פונקציית האובדן מטילה עליו קנס גדול. אין לחשב לוגריתם של אפס ללא טיפול מספרי מתאים; ספריות מקצועיות מבצעות חישובים יציבים יותר.

7. העברת נגזרות לאחור — Backpropagation

עד עכשיו ידענו לחשב תחזית, אבל עדיין לא ידענו אילו משקולות לשנות. Backpropagation הוא שיטה יעילה לחישוב הנגזרת של פונקציית האובדן ביחס לכל פרמטר ברשת. הוא מתקדם לאחור דרך פעולות הרשת תוך שימוש בכלל השרשרת, ומנצל תוצאות שנשמרו במעבר הקדמי. לאחר שחושבו הנגזרות, האלגוריתם המעדכן את המשקולות — למשל Gradient Descent — מבצע את השינוי בפועל.



Backward pass: chain-rule gradients propagate towards earlier layers

איור: כיוון חישוב הנגזרות: מן השגיאה אל שכבות הקלט.

בפרק 11 למדנו את כלל השרשרת: הנגזרת של פונקציה מורכבת היא מכפלת הנגזרות המקומיות לאורך המסלול. עכשיו נראה כיצד הכלל הופך לתהליך עבודה מסודר בנוירונים אחדים.

dL/do = o - y = -0.34

ראשית מחשבים את הנגזרת ביחס לפלט. במקרה שלנו היא שלילית, ולכן הגדלת הפלט מעט צפויה להקטין את השגיאה בסביבת המצב הנוכחי. נגזרת זו מועברת אל המשקולות בשכבת הפלט דרך הקשר בין הפלט לערכי השכבה החבויה.

dL/dW2 = (o-y) * h = (-0.374, -0.170)

dL/db2 = o-y = -0.34

המשקל הראשון בשכבת הפלט קיבל נגזרת −0.374 כי ערך הנוירון המתאים הוא 1.1. המשקל השני קיבל −0.170 כי ערך הנוירון השני 0.5. זוהי דוגמה לכך שהנוסחה נגזרת מהתלות של כל משקל בחישוב, ולא מכלל שרירותי של שינוי מספרים.

delta_hidden = (o-y) * W2 * ReLU'(z1)

delta_hidden = (-0.204, +0.136)

שני הערכים z_1 חיוביים, ולכן נגזרת ReLU היא אחת בכל אחד מהם. המשקל השני בשכבת הפלט שלילי, ולכן סימן הנגזרת המוחזרת לנוירון החבוי השני מתהפך. הדבר חשוב: לא כל משקל צריך להשתנות באותו כיוון.

dL/dW1 = outer(delta_hidden, x)

dL/dW1 = [[-0.204, -0.408], [0.136, 0.272]]

dL/db1 = (-0.204, 0.136)

סיימנו לחשב את הגרדיאנטים. יש לנו נגזרת לכל משקל ולכל הטיה, ומכאן ניתן לבצע עדכון. המילה outer מציינת כאן מכפלה חיצונית בין וקטור נגזרות הנוירונים לבין וקטור המאפיינים.

8. עדכון אחד מלא של המשקולות

נבחר קצב למידה $\eta = 0.1$. בכל פרמטר מחסרים את קצב הלמידה כפול הנגזרת שלו. אם הנגזרת שלילית — הערך יגדל; אם הנגזרת חיובית — הערך יקטן. העדכון אינו מבטיח ירידה בשגיאה בכל צעד ובכל קצב למידה, אך בדוגמה הקטנה שלנו נוכל לחשב ולבדוק את השפעתו.

parameter_new = parameter_old - 0.1 * gradient

פרמטר	לפני העדכון	נגזרת	לאחר העדכון
W2[0]	0.6000	-0.3740	0.6374
W2[1]	-0.4000	-0.1700	-0.3830
b2	0.2000	-0.3400	0.2340
W1[0,0]	0.2000	-0.2040	0.2204
W1[0,1]	0.4000	-0.4080	0.4408
W1[1,0]	-0.3000	+0.1360	-0.3136
W1[1,1]	0.5000	+0.2720	0.4728

פרמטר	לפני העדכון	נגזרת	לאחר העדכון
b1[0]	0.1000	-0.2040	0.1204
b1[1]	-0.2000	+0.1360	-0.2136

כאשר מחשבים מעבר קדימה נוסף אחרי העדכון, מקבלים תחזית בקירוב 0.8529, ושגיאה חדשה 0.01082 במקום 0.0578. זאת הדגמה של צעד למידה אחד על תצפית אחת בלבד. עדיין לא נוכל להסיק ממנה שהמודל מצליח על נתונים שלא ראה.

9. מעבר מחישוב ידני לקוד NumPy

נכתוב קוד הניתן להרצה רצופה הממחיש את אותם חישובים. הקוד מציג את הנגזרות ולא מסתיר אותן בפעולת fit. בהמשך, ספריית למידה עמוקה תחשב אותן אוטומטית, אבל ההבנה המתמטית שכאן מאפשרת להסביר מה הכלים עושים.

```
import numpy as np
x = np.array([1.0, 2.0])
y = 1.0
W1 = np.array([[0.2, 0.4], [-0.3, 0.5]])
b1 = np.array([0.1, -0.2])
W2 = np.array([0.6, -0.4])
b2 = 0.2
eta = 0.1

z1 = W1 @ x + b1
h = np.maximum(0, z1)
o = W2 @ h + b2
loss_before = 0.5 * (o - y)**2

delta_o = o - y
gW2 = delta_o * h
gb2 = delta_o
delta_h = delta_o * W2 * (z1 > 0)
gW1 = np.outer(delta_h, x)
gb1 = delta_h

W2 -= eta * gW2
b2 -= eta * gb2
W1 -= eta * gW1
b1 -= eta * gb1

h_after = np.maximum(0, W1 @ x + b1)
o_after = W2 @ h_after + b2
loss_after = 0.5 * (o_after - y)**2
print(round(loss_before, 5), round(loss_after, 5))
```

שימו לב לשלושה רעיונות בקוד: הווקטור z1 נוצר לפני ההפעלה; הביטוי `(z1 > 0)` מחזיר מסכה המתאימה לנגזרת ReLU במקרה הנלמד; ולבסוף מעדכנים את כל המשקולות רק אחרי שחושבו הנגזרות מתוך אותו מצב התחלתי. שינוי משקל מוקדם מדי במהלך החישוב עלול לערבב מצבים שונים ולשבש את התוצאה.

10. חישוב ממדים — כמה פרמטרים יש לרשת?

הצורה של מטריצת משקולות נקבעת לפי מספר הנוירונים בשכבות. אם בשכבה קודמת 3 נוירונים ובשכבה הבאה 4 נוירונים, דרושים 12 משקולות לחיבורים ועוד 4 הטיות. לכן מספר הפרמטרים של מעבר בין שכבות הוא מכפלת גדלי השכבות בתוספת מספר הנוירונים בשכבה הבאה.

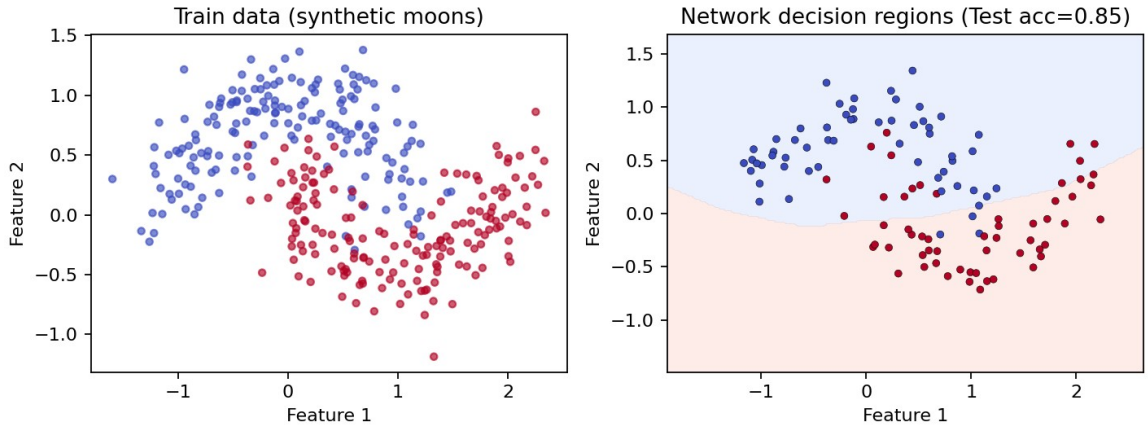
Parameters = n_in * n_out + n_out

לרשת בעלת 4 מאפייני קלט, 5 נוירונים חבויים, 3 נוירונים חבויים בשכבה השנייה ו־2 נוירוני פלט: בין קלט לשכבה הראשונה יש 25 פרמטרים; בין השכבות החבויות 18; ובין השנייה לפלט 8. סך הכול 51 פרמטרים. חשבו זאת ביד לפני הפעלת תוכנה.

יותר פרמטרים מאפשרים לייצג פונקציות מורכבות יותר, אך אינם מבטיחים ביצועים טובים יותר. רשת גדולה דורשת יותר נתונים, חישוב ואמצעי בקרה למניעת התאמת יתר.

11. מרשת קטנה למימוש מקצועי ב־scikit-learn

כדי לראות רשת לומדת על נתונים מורכבים מעט יותר, נשתמש במאגר סינתטי שנראה כשתי קשתות. זהו מאגר לימודי, ולא נתונים אמיתיים על בני אדם או על תהליך עסקי. נפריד את הנתונים לאימון ולבדיקה, ונבנה Pipeline המתקנן את המאפיינים באמצעות נתוני האימון בלבד.



איור: דוגמת שתי קשתות: רשת עם שכבות חבויות יכולה ללמוד אזורי החלטה שאינם קו ישר.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

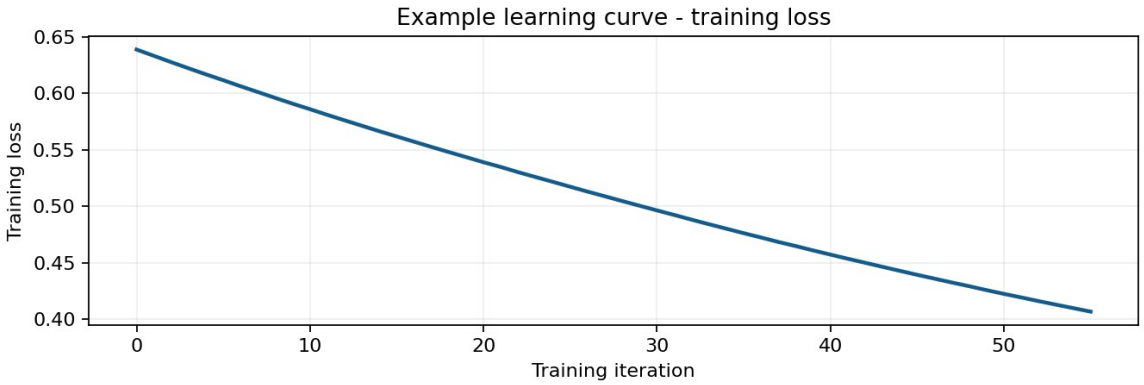
X, y = make_moons(n_samples=450, noise=0.22, random_state=8)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=15
)
model = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(8, 8),
                  max_iter=600, early_stopping=True,
                  n_iter_no_change=25, random_state=12)
)
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
print("Test accuracy:", accuracy_score(y_test, y_pred))
```

הארכיטקטורה `(8, 8)` פירושה שתי שכבות חבויות, שבכל אחת שמונה נוירונים. המשתנה `max_iter` מגביל את מספר איטרציות האימון לפי המימוש של הספרייה. `early_stopping=True` מאפשר עצירה מוקדמת על בסיס ולידציה פנימית. הדוגמה מיועדת להבנת הצינור המקצועי; היא אינה מחליפה בחירה מושכלת של ארכיטקטורה והערכת תוצאות על בעיה אמיתית.

12. כיצד קוראים עקומת למידה?

במהלך האימון המודל מציג לעיתים את שינוי פונקציית האובדן לאורך איטרציות. אם השגיאה באימון יורדת, סימן שהתהליך מצליח להתאים טוב יותר את הדוגמאות ששימשו לאימון. אך אין בכך הוכחה ליכולת הכללה. נדרשת גם בדיקה של ביצועים על נתוני ולידציה ובסוף על קבוצת Test שלא שימשה לבחירות.

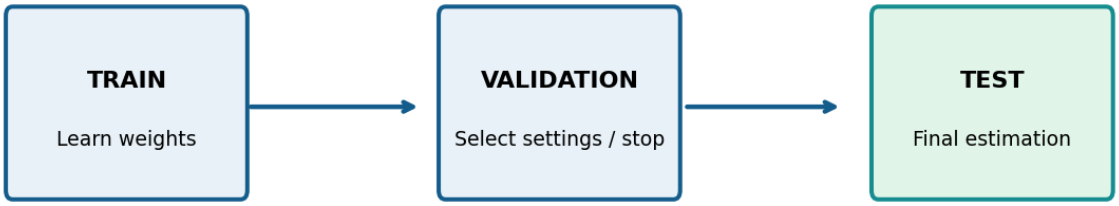


איור: עקומת שגיאת אימון מתוך הניסוי הסינתטי; לא להסיק ממנה לבדה על הכללה.

למידת חסר מתאפיינת לעיתים בשגיאה גבוהה גם באימון וגם בוולידציה, בעוד התאמת יתר יכולה להתבטא בפער גדל ביניהן. יש להיזהר מלהפוך סימנים אלה לכללים מוחלטים: אופי הנתונים, מדד השגיאה ומידת הרעש משפיעים על צורת העקומות.

13. Train, Validation ו־Test — מדוע לא משתמשים באותם נתונים לכל דבר?

אימון משמש לעדכון משקולות. ולידציה משמשת לבחירת ארכיטקטורה, קצב למידה, מספר Epochs או נקודת עצירה. Test מיועד להערכה סופית אחרי שהתקבלו ההחלטות. אם בוחרים את מבנה הרשת שוב ושוב לפי תוצאות Test, המבחן אינו בלתי תלוי עוד, והתוצאה המדווחת עלולה להיות אופטימית.



איור: שלוש מטרות שונות לשימוש בנתונים. *Pipeline* מפחית סיכון לדליפת מידע.

דוגמה לזליגה: מחשבים ממוצע וסטיית תקן על כלל המאגר לפני חלוקתו, ואז משתמשים בהם בתקנון נתוני האימון. כך מידע על נתוני הבדיקה השפיע בעקיפין על האימון. דרך נכונה היא להתאים את התקנון לנתוני האימון בלבד. בניסויי Cross-Validation יש להתאים את התקנון מחדש בתוך כל קיפול אימון.

14. אופטימיזציה, Batch ו־Epoch

כאשר מאגר הנתונים גדול, חישוב נגזרות על כל התצפיות בכל צעד עלול להיות יקר. לכן נהוג לעבד קבוצות קטנות (mini-batches). בכל Batch מחשבים תחזיות, אובדן, גרדיאנטים ועדכון. Epoch הוא מעבר מלא על דוגמאות האימון; אם גודל האימון 1,000 וגודל Batch הוא 100, יהיו בדרך כלל עשרה צעדי Batch בכל Epoch, אם לא משנים את דוגמת החלוקה.

קצב הלמידה קובע את אורך העדכון. אם הוא קטן מאוד, השיפור עשוי להיות איטי; אם הוא גדול מדי, המודל עלול להחמיץ אזורי שגיאה נמוכה או להפוך את האימון לבלתי יציב. שיטות כגון SGD ו־Adam משנות את אופן ביצוע העדכונים, אך אינן מחליפות את העיקרון של חישוב גרדיאנטים.

15. מה הקשר בין Backpropagation ל־Automatic Differentiation?

ברשתות מודרניות אין גוזרים ביד מאות אלפי משקולות. מסגרות כגון PyTorch בונות גרף חישובי ומשתמשות בגזירה אוטומטית לחישוב גרדיאנטים. הפעולה `loss.backward()` מחשבת נגזרות; היא אינה מעדכנת לבדה את המשקולות. לאחריה פעולה כגון `optimizer.step()` מבצעת עדכון. לפני צעד האימון הבא יש לאפס גרדיאנטים צבורים, למשל באמצעות `optimizer.zero_grad()`.

> הרחבה מקצועית: דוגמת *PyTorch* היא גשר להמשך ולא דרישה שכל תלמיד יכתוב את תשתית *Autograd* בעצמו. השימושים בקוד מקצועי מחייבים הסבר של מבנה המודל והחלטות האימון.

```
# A short professional pattern (PyTorch installed separately)
# optimizer.zero_grad()
# prediction = model(features)
# loss = loss_fn(prediction, target)
# loss.backward()       # compute gradients
# optimizer.step()      # update parameters
```

16. תקלות נפוצות וכיצד מאבחנים אותן

תצפית	מה ייתכן שקרה?	כיצד נבדוק?
שגיאת אימון אינה יורדת	קצב למידה, תקנון, נתונים או באג בקוד	בדיקת טווחים, גרדיאנטים וקו בסיס
אימון מצוין, Test חלש	התאמת יתר או זליגת מידע בבחירת מודל	השוואת עקומות ואופן חלוקת נתונים
פלט כמעט קבוע	ארכיטקטורה לא מתאימה או רשת שאינה לומדת	בדיקת משקולות, קלטים ופלטים
שגיאה מספרית NaN	ערכים קיצוניים, קצב למידה או לוגריתם אפס	ניטור חישובים ותקנון
מדדים משתנים בין הרצות	אקראיות באתחול ובחלוקה	זרעים קבועים וניסוי חוזר
פלט בינארי אינו 0/1	המודל מחזיר ציון או הסתברות לפני סף	הגדרת כלל החלטה מפורש

17. מדוע גם רשת מצליחה יכולה להיות בעייתית?

רשת עצבית היא כלי חיזוי סטטיסטי, ולא מכשיר הקובע אמת מוחלטת. אם הנתונים אינם מייצגים אוכלוסיות מסוימות, אם נאספו באופן לא מוסכם, או אם התוויות אינן אמינות — איכות גבוהה במדד מספרי יחיד אינה מספיקה כדי להצדיק שימוש. בפרויקט נדרש לתאר את מקור הנתונים, את מגבלותיהם, את השגיאות ואת הסיכון הנובע מן השימוש במודל.

בתחומים רגישים, כגון בריאות, מיון תלמידים או קבלת החלטות בעלות השלכות משמעותיות, יש צורך בבקרה מקצועית ובשמירה על פרטיות. את פרק 12 ניישם בבעיות לימודיות שניתן לבקר ולהסביר, ולא נציג תחזית אוטומטית כאבחנה או כפסיקה.

18. תרגול מדורג — מהבנה לעומק

רמה א — מושגים וחישוב: (א) הסבירו במילים שלכם את ההבדל בין שכבת קלט לשכבה חבויה. (ב) חשבו עבור ReLU $2, 0-$ ו־3. (ג) ברשת בעלת 2 מאפיינים, 3 נוירונים חבויים ו־1 פלט — כמה משקולות וכמה הטיות יש? הציגו שלבי חישוב.

רמה ב — העברה ונימוק: (ד) נתונה רשת שבה כל השכבות ליניאריות. הסבירו למה הוספת שכבה לא בהכרח תפתור XOR. (ה) במודל שבדוגמה הידנית, מה יקרה לערך הנוירון השני אם נשנה את אחת המשקולות? נמקו בחישוב. (ו) מדוע אי־אפשר לפרש תחזית 0.66 בהכרח כהסתברות?

רמה ג — פיתוח וביקורת: (ז) גזרו מחדש נגזרת של משקל אחד בשכבה החבויה באמצעות כלל השרשרת. (ח) הציעו ניסוי שמשווה רשת בעלת שכבה חבויה אחת לרשת בעלת שתי שכבות, באותה חלוקת נתונים. (ט) תארו שלושה צעדים למניעת זליגת מידע בפרויקט אמיתי.

19. שאלת הערכה מקורית בסגנון בגרות / הגנה

מערכת AI מקבלת שני מאפיינים של מוצר ומפיקה תחזית מספרית. המודל כולל שכבה חבויה אחת בת שני נוירונים עם ReLU ושכבת פלט ליניארית. נתונים המשקולות, ההטיות והקלט מהדוגמה בסעיף 5. התלמיד מתבקש: (א) לחשב את פלט השכבה החבויה; (ב) לחשב תחזית ושגיאה; (ג) להסביר איזה שלב

נדרש כדי לעדכן משקולות; (ד) להציע בדיקה אחת שתבחן יכולת הכללה; (ה) לנמק מדוע הרשת עשויה להיות מתאימה יותר ממודל ליניארי פשוט עבור תופעה לא־ליניארית.

הערה: זוהי שאלה מקורית בדפוס הערכה עיוני והגנה בעל־פה, ואינה שאלה רשמית מתוך בחינת משרד החינוך.

20. רצף הוראה מוצע — 20 שעות

מפגש	שעות הוראה	מוקד	ראיה ללמידה
1	2	בעיה אמיתית, פרספטרון לעומת MLP, XOR	המחשת מגבלת ליניאריות
2	2	שכבות, משקולות, הטיות וממדים	שירטוט והערכת פרמטרים
3	2	פונקציות הפעלה ו־ReLU	חישוב ערכי הפעלה
4	2	Forward Pass ידני	טבלת עקיבה
5	2	פונקציות אובדן ורגרסיה/סיווג	חישוב Loss ופירוש
6	2	Backpropagation וכלל השרשרת	טבלת גרדיאנטים
7	2	עדכון משקולות ו־NumPy	מימוש חישוב ידני
8	2	אימון רשת וספריית sklearn	ניסוי עם שני ארכיטקטורות
9	2	Evaluation, Validation, Leakage	דוח ניסוי מנומק
10	2	תרגול עצמאי, שאלת הערכה והכנה לפרויקט	מחברת, מסקנה והסבר בעל־פה

חלוקת עשר יחידות לשתי שעות היא הצעה דידקטית מתוך היקף 20 השעות של פרק ההתמחות, ולא חלוקה רשמית של משרד החינוך.

21. מילון מושגים קצר

מושג	פירוש
MLP	רשת נוירונים קדמית בעלת כמה שכבות
Forward Pass	מעבר חישוב מהקלט אל הפלט
Backpropagation	חישוב גרדיאנטים לאחור באמצעות כלל השרשרת
Activation	פונקציה שמשנה את פלט הנוירון
Loss	פונקציית שגיאה לאימון
Gradient	וקטור הנגזרות של פונקציית האובדן
Optimizer	כלל עדכון פרמטרים

מושג	פירוש
Epoch	מעבר מלא על נתוני אימון
Batch	קבוצת תצפיות בתוך האימון
Early Stopping	עצירה לפי התקדמות מדד ולידציה

22. הקשר בין שלוש תכניות הלימוד

תכנית	מה כבר נלמד?	מה מוסיף פרק זה?
יחידה 3	Python, וקטורים בסיסיים, סיווג והערכה	חיבור לרשת כאמצעי ייצוג חדש — גשר בלבד
יחידה 5 נושאי ליבה	וקטורים, מרחקים, סטטיסטיקה ואימות מודלים	ניצול המתמטיקה שכבר נלמדה, ללא ייחוס רשתות כחובת יחידה 5
התמחות DL	רגרסיה, פרספטרון, נגזרות ו־Gradient Descent	מעבר קדימה, Backpropagation ורשת מאומנת

23. ספרות מקצועית ומקורות

עוגן תכנית: משרד החינוך, התמחות למידת מכונה ולמידה עמוקה, פרק 12 — רשת נוירונים עמוקה (20 שעות), על פי מפת התכנית השמורה במאגר GitHub. פירוט הדוגמאות, תרגילי ההעשרה והחלוקה למפגשים הם פיתוח מקורי; האימות הסופי של תתי־סעיפי חובה מחייב בדיקת מסמך התכנית הרשמי המעודכן.

Zhang, A., Lipton, Z., Li, M. & Smola, A. — Dive into Deep Learning, פרק Multilayer Perceptrons ובפרט Forward / Backpropagation: https://d2l.ai/chapter_multilayer-perceptrons/backprop.html

Prince, S. J. D. — Understanding Deep Learning, MIT Press, 2023. Deisenroth, Faisal & Ong — Mathematics for Machine Learning, Cambridge University Press, 2020. Géron — Hands-On Machine Learning. המקורות משמשים לעקרונות, לרמת ההסבר ולביקורת, ולא הועתקו מהם איורים או קטעי טקסט.

scikit-learn — User Guide: Neural Network Models (Supervised): https://scikit-learn.org/stable/modules/neural_networks_supervised.html. PyTorch — Neural Networks Tutorial: https://docs.pytorch.org/tutorials/beginner/blitz/neural_networks_tutorial